

SMART MOBILITY

Analisi Tecnica per User Story

Logica di Sistema · Mapping Funzioni–Codice · Percorso Dati nel Database

Progetto	Smart Mobility — Gruppo C--
Versione Documentazione	6.0 — 28/06/2026
Corso	Ingegneria del Software A.A. 2025-2026
Team	Andriani Chiara · Andriani Claudio · Azzollini Vito · Campi Mattia
User Stories analizzate	35 (UT.01-18 · AP.01-07 · OP.01-10)
Database	PostgreSQL 15 — schema: public

UTENTE

AMMINISTRAZIONE

OPERATORE

● UTENTE (18 User Stories)

- UT.01 / CU-02 — Visualizzazione mappa mezzi disponibili
- UT.02 / CU-03 — Prenotazione di un mezzo
- UT.03 / CU-06 — Ricerca destinazione
- UT.04 / CU-07 — Stima del costo del viaggio
- UT.05 / CU-05 — Termine corsa
- UT.06 / CU-08 — Percorso ottimale in tempo reale
- UT.07 / CU-09 — Suggerimento mezzo piu adatto
- UT.08 / CU-10 — Visualizzazione e applicazione promozioni
- UT.09 / CU-11 — Chat con il servizio clienti
- UT.10 / CU-12 — Login utente
- UT.11 / CU-13 — Visualizzazione stato di carica
- UT.12 / CU-04 — Sblocco mezzo tramite QR code o app
- UT.13 / CU-15 — Gestione metodo di pagamento
- UT.14 / CU-14 — Pausa corsa
- UT.15 / CU-01 — Creazione profilo utente
- UT.16 / CU-16 — Visualizzazione aree non accessibili
- UT.17 / CU-17 — Segnalazione problematiche percorso
- UT.18 / CU-18 — Pagamento corsa

● AMMINISTRAZIONE (7 User Stories)

- AP.01 / CU-19 — Monitoraggio frequenza utilizzo mezzi
- AP.02 / CU-20 — Segnalazione manutenzioni urbane
- AP.03 / CU-21 — Tratte piu utilizzate
- AP.04 / CU-22 — Configurazione aree con restrizioni
- AP.05 / CU-23 — Report aggregati sulla mobilita
- AP.06 / CU-24 — Registrazione Amministrazione Pubblica
- AP.07 / CU-25 — Login Amministrazione Pubblica

● OPERATORE (10 User Stories)

- OP.01 / CU-26 — Registrazione Operatore
- OP.02 / CU-29 — Notifiche aree a diversa disponibilita
- OP.03 / CU-30 — Conoscenza malfunzionamenti mezzi
- OP.04 / CU-31 — Posizione mezzo a fine corsa
- OP.05 / CU-32 — Tracciamento posizione in tempo reale
- OP.06 / CU-33 — Chiusura segnalazione
- OP.07 / CU-34 — Assegnazione automatica bonus
- OP.08 / CU-35 — Sospensione o blocco account utente
- OP.09 / CU-28 — Blocco remoto mezzo
- OP.10 / CU-27 — Login Operatore

Visualizzazione mappa mezzi disponibili

Endpoint	Screen / Componente
GET /vehicles?lat=&lng;=&only;_available=true	(app)/index.tsx

Descrizione Logica del Sistema

All'apertura dell'app la funzione **list_vehicles()** in *vehicles.py* viene chiamata con le coordinate GPS dell'utente. Il sistema recupera tutti i veicoli dal database e applica un filtro: vengono mostrati solo quelli con **status='parked'** e **locked=False**, cioè fisicamente disponibili e non bloccati dall'operatore. Se nelle vicinanze ci sono meno di 12 veicoli entro circa 1,2 km, la logica **spawn-near-user** sposta automaticamente alcuni veicoli lontani nelle 8 aree di parcheggio più vicine all'utente tramite la funzione ausiliaria **_rand_coord_in_area()**, aggiornando le coordinate nel DB. Parallelamente, la funzione **list_aree()** in *restrizioni.py* fornisce i poligoni delle zone vietate che vengono sovrapposti alla mappa come overlay colorati.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
vehicles	SELECT	PostgreSQL → schema: public → tabella: vehicles	id, name, type, lat, lng, battery_pct, status, locked Filtro: status='parked' AND locked=false
vehicles	UPDATE	PostgreSQL → schema: public → tabella: vehicles	lat, lng Solo se spawn-near-user attivo (<12 veicoli vicini)
parking_areas	SELECT	PostgreSQL → schema: public → tabella: parking_areas	id, name, lat, lng, radius_m Lettura delle 8 aree più vicine per spawn
aree_restriz ione	SELECT	PostgreSQL → schema: public → tabella: aree_restrizione	id, nome, tipo, lat, lng, radius_m, attiva Overlay zone vietate sulla mappa

Come Verificare

Aprire la mappa: contare i pin visibili (min 12 vicini). Verificare in DB: SELECT * FROM vehicles WHERE locked=true → nessuno deve comparire sulla mappa utente.

Prenotazione di un mezzo

Endpoint	Screen / Componente
POST /prenotazioni GET /prenotazioni/attiva DELETE /prenotazioni/{id}	(app)/reserve.tsx + (app)/active-reservation.tsx

Descrizione Logica del Sistema

Quando l'utente sceglie un mezzo e tappa 'Prenota', la funzione **create()** in *reservations.py* verifica prima che l'utente non abbia già una prenotazione attiva (restituisce 409 se presente). Controlla poi che il mezzo selezionato esista nel DB; se il suo status risulta non disponibile ma non c'è nessuna corsa attiva associata, lo reimposta automaticamente ad 'available' per gestire stati orfani. Crea quindi un record **Reservation** con scadenza a 10 minuti dal momento attuale (costante **RESERVATION_DURATION**). L'utente viene indirizzato alla schermata **active-reservation.tsx** che mostra un countdown. La funzione **get_active()** verifica in ogni momento se la prenotazione è ancora valida confrontando **expires_at** con il timestamp corrente.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
reservations	INSERT	PostgreSQL → schema: public → tabella: reservations	user_id, vehicle_id, created_at, expires_at, status status='active', expires_at=now+10min
reservations	SELECT	PostgreSQL → schema: public → tabella: reservations	user_id, status, expires_at Controllo duplicata + GET /prenotazioni/attiva
vehicles	SELECT	PostgreSQL → schema: public → tabella: vehicles	id, status Verifica disponibilit� mezzo
vehicles	UPDATE	PostgreSQL → schema: public → tabella: vehicles	status Solo se stato orfano: reset a 'available'

Come Verificare

POST /prenotazioni: verificare INSERT in reservations (status='active'). Tentare seconda prenotazione → 409. Attendere 10 min: expires_at scaduto, GET /prenotazioni/attiva → null.

Ricerca destinazione

Endpoint	Screen / Componente
GET /geocode?q= GET /route/options?from_lat=&from;_lng=&to;_lat=&to;_lng=&vehicle;_type=	(app)/search.tsx

Descrizione Logica del Sistema

Mentre l'utente digita nel campo di ricerca, la funzione **geocode()** in *geo.py* interroga in tempo reale le API di **Nominatim OpenStreetMap** restituendo fino a 5 suggerimenti di indirizzo con le relative coordinate. Una volta selezionata la destinazione, la funzione **route_options()** chiama il servizio **OSRM** per ottenere fino a 3 percorsi alternativi. Per ognuno, la funzione **_aree_attraversate()** controlla se il tracciato passa attraverso zone di restrizione attive recuperate dalla tabella **aree_restrizione**. I percorsi vengono ordinati: prima quelli senza violazioni, poi per distanza crescente. Il primo è etichettato 'Percorso consigliato'.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
aree_restrizione	SELECT	PostgreSQL → schema: public → tabella: aree_restrizione	id, nome, tipo, lat, lng, radius_m, vehicle_types, attiva <i>Lettura zone attive per controllo percorso (route_options)</i>

Come Verificare

GET /geocode?q=Bari: ricevere 5 risultati Nominatim. GET /route/options con percorso che attraversa area NO_GO: il campo restricted=true nel JSON di risposta.

Stima del costo del viaggio

Endpoint	Screen / Componente
GET /route/options (usa duration_min + dati veicolo)	(app)/search.tsx

Descrizione Logica del Sistema

Non esiste un endpoint dedicato per il calcolo: il costo stimato viene calcolato direttamente in **search.tsx** sul client. La logica usa il campo **duration_min** restituito da **route_options()** — calcolato in **geo.py** come $\text{distanza_km} / \text{velocita_media} * 60$ — moltiplicato per il **price_per_min** del veicolo selezionato (0.22 €/min per scooter, variabile per gli altri), più la quota fissa **unlock_fee** (1.00 € di default). Il risultato è mostrato come stima indicativa; il costo definitivo viene calcolato server-side al termine della corsa dalla funzione **end_ride()** in **rides.py** per evitare manomissioni lato client.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
vehicles	SELECT	PostgreSQL → schema: <code>public</code> → tabella: <code>vehicles</code>	unlock_fee, price_per_min <i>Lettura tariffe dal payload del veicolo selezionato</i>

Come Verificare

Percorso 2 km con scooter: $\text{duration_min} \approx 6.6 \text{ min} \rightarrow \text{costo stimato} \approx 1.00 + 6.6 * 0.22 = 2.45$ €. Confrontare con costo reale post-corsa da **rides.cost** nel DB.

Termine corsa

Endpoint	Screen / Componente
POST /rides/{id}/end PATCH /vehicles/{id}/park	(app)/end-ride.tsx + (app)/ride-payment.tsx

Descrizione Logica del Sistema

Quando l'utente termina la corsa, la funzione **end_ride()** in *rides.py* calcola la durata server-side confrontando il timestamp attuale con **ride.started_at** memorizzato nel DB — il client non può alterare questo valore. Il costo viene calcolato come **unlock_fee + minuti * price_per_min**, addebitato sul saldo utente (**users.balance**) e registrato come transazione negativa in **wallet_transactions**. Il mezzo cambia stato in 'available'. Successivamente l'utente sceglie un'area di parcheggio: la funzione **park_vehicle()** in *vehicles.py* posiziona il mezzo dentro il raggio dell'area con coordinate casuali via **_rand_coord_in_area()** e imposta **status='parked'**. Viene inserito un messaggio di conferma nel sistema.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
rides	UPDATE	PostgreSQL → schema: public → tabella: rides	status, km, minutes, cost, points, ended_at <i>status='completed', costo calcolato server-side</i>
vehicles	UPDATE	PostgreSQL → schema: public → tabella: vehicles	status, lat, lng <i>status='parked', coordinate dentro raggio area</i>
users	UPDATE	PostgreSQL → schema: public → tabella: users	balance, points <i>balance -= cost; points += 10 (+5 eco se km >= 2)</i>
wallet_transactions	INSERT	PostgreSQL → schema: public → tabella: wallet_transactions	user_id, type, label, amount, created_at <i>type='charge', amount=-cost (negativo)</i>
parking_areas	UPDATE	PostgreSQL → schema: public → tabella: parking_areas	occupied <i>occupied += 1 se non a capienza massima</i>
messages	INSERT	PostgreSQL → schema: public → tabella: messages	user_id, type, title, body <i>type='ride', titolo 'Corsa terminata'</i>

Come Verificare

Dopo **end_ride**: SELECT status FROM rides WHERE id=X → 'completed'. SELECT balance FROM users WHERE id=Y → diminuito. SELECT amount FROM wallet_transactions ORDER BY created_at DESC LIMIT 1 → negativo.

Percorso ottimale in tempo reale

Endpoint	Screen / Componente
GET /route/options GET /route	(app)/search.tsx + (app)/active-ride.tsx

Descrizione Logica del Sistema

La funzione **route_options()** in *geo.py* chiama OSRM con il parametro **alternatives=3** per ottenere più percorsi. Per ogni alternativa calcola il tempo stimato in base alla velocità media del mezzo (18 km/h scooter, 20 ebike, 30 auto) e verifica se il tracciato attraversa zone vietate tramite **_aree_attraversate()**. I percorsi senza restrizioni vengono proposti per primi. Durante la corsa, **active-ride.tsx** usa la funzione **route()** per ottenere la polilinea completa e la ridisegna sulla mappa ad ogni cambio di posizione GPS dell'utente, simulando la navigazione in tempo reale.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
aree_restrizio ne	SELECT	PostgreSQL → schema: public → tabella: aree_restrizione	tipo, lat, lng, radius_m, vehicle_types, attiva Controllo restrizioni per ogni punto del percorso OSRM

Come Verificare

Con area NO_GO attiva sul percorso diretto: il primo risultato deve avere **restricted=false** (percorso alternativo). Rimuovere l'area: il percorso diretto torna primo.

Suggerimento mezzo piu adatto

Endpoint	Screen / Componente
GET /vehicles?only_available=true	(app)/search.tsx

Descrizione Logica del Sistema

La logica di suggerimento risiede interamente in **search.tsx** sul client, senza endpoint dedicato. Dopo aver ottenuto distanza e tipo di percorso da **route_options()**, la schermata filtra i veicoli disponibili in base al campo **battery_pct** ricevuto da **list_vehicles()**: i mezzi con autonomia stimata insufficiente a coprire la distanza vengono ordinati in fondo e mostrano un avviso visivo. Il calcolo dell'autonomia è approssimativo (basato su **battery_pct** e tipo mezzo) e serve come guida per l'utente, non come blocco rigido.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
vehicles	SELECT	PostgreSQL → schema: public → tabella: vehicles	id, type, battery_pct, lat, lng <i>Lettura autonomia (battery_pct) per ordinamento suggerimenti</i>

Come Verificare

Impostare **battery_pct=5** per un veicolo via DB: deve comparire in fondo alla lista con badge di avviso. **battery_pct=100**: deve essere proposto come primo.

Visualizzazione e applicazione promozioni

Endpoint	Screen / Componente
GET /promotions POST /promotions/redeem	(app)/promotions.tsx + (app)/ride-payment.tsx

Descrizione Logica del Sistema

La funzione **list_promotions()** in *promotions.py* restituisce tutte le promozioni presenti in DB ordinate per tipo e id. In **promotions.tsx** l'utente visualizza coupon attivi e scaduti. Per applicare uno sconto, la funzione **redeem_promotion()** cerca il codice nella tabella **promotions** (convertito in maiuscolo), verifica che non sia scaduto (confronto con data odierna) e che non sia esaurito ($used < total$). Se valido, incrementa il contatore **used** e inserisce una notifica nella inbox dell'utente. Lo sconto viene applicato al calcolo del costo finale in *ride-payment.tsx*.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
promotions	SELECT	PostgreSQL → schema: public → tabella: promotions	id, code, title, body, reward, expiry, used, total Lettura lista promozioni e verifica codice
promotions	UPDATE	PostgreSQL → schema: public → tabella: promotions	used used += 1 al momento del riscatto
messages	INSERT	PostgreSQL → schema: public → tabella: messages	user_id, type, title, body type='promo', notifica riscatto in inbox

Come Verificare

SELECT used FROM promotions WHERE code='PROMO123' prima e dopo il riscatto → deve incrementarsi. Riscattare codice esaurito (used=total): risposta 400.

Chat con il servizio clienti

Endpoint	Screen / Componente
GET /messages POST /messages POST /messages/read-all	(app)/support.tsx + (app)/chat-support.tsx

Descrizione Logica del Sistema

Il sistema di messaggistica usa un'unica tabella **messages** per tutti i tipi di comunicazione (promo, ride, alert, system). La funzione **list_messages()** in *messages.py* recupera tutti i messaggi dell'utente ordinati per data decrescente. Quando l'utente invia un messaggio al supporto, **create_message()** inserisce un record con **type='system'**. Il campo **read** traccia i messaggi non letti: la funzione **mark_all_read()** aggiorna in batch tutti i messaggi non letti con una singola query UPDATE, e il badge del contatore nel layout si azzerà di conseguenza.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
messages	SELECT	PostgreSQL → schema: public → tabella: messages	id, user_id, type, title, body, read, created_at Filtro: user_id = utente corrente, ORDER BY created_at DESC
messages	INSERT	PostgreSQL → schema: public → tabella: messages	user_id, type, title, body Nuovo messaggio utente o notifica di sistema
messages	UPDATE	PostgreSQL → schema: public → tabella: messages	read read=true su tutti i messaggi non letti (mark-all-read)

Come Verificare

SELECT COUNT(*) FROM messages WHERE user_id=X AND read=false prima di mark-all-read → N. Dopo: 0. POST /messages: verificare INSERT con type='system'.

Login utente

Endpoint	Screen / Componente
POST /auth/login GET /auth/me	(auth)/login.tsx

Descrizione Logica del Sistema

La funzione **login()** in *auth.py* riceve email e password, interroga la tabella **users** per email e verifica la password con **verify_password()** usando bcrypt (da *security.py*). Se le credenziali sono corrette ma **account_status** è 'SOSPESO' o 'BLOCCATO', il sistema restituisce 403 Forbidden impedendo l'accesso. In caso di successo, **create_access_token()** genera un JWT firmato con il segreto **JWT_SECRET** che include **user_id** e **role** nel payload. Il token viene salvato in **SecureStore** (storage cifrato del dispositivo) e incluso nell'header **Authorization: Bearer** di ogni richiesta successiva.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
users	SELECT	PostgreSQL → schema: public → tabella: users	id, email, password_hash, role, account_status <i>Query per email; nessuna scrittura al login (JWT stateless)</i>

Come Verificare

Login con password errata → 401. Login con **account_status='BLOCCATO'** → 403. Login OK: decodificare JWT con **JWT_SECRET** → verificare claim **user_id** e **role**.

Visualizzazione stato di carica

Endpoint	Screen / Componente
GET /vehicles/{id}	(app)/index.tsx (pannello dettaglio mezzo)

Descrizione Logica del Sistema

Il campo **battery_pct** della tabella **vehicles** è incluso in ogni risposta di **list_vehicles()** e **get_vehicle()** in *vehicles.py*. La schermata principale mostra una barra di progresso colorata in base alla carica: verde se `battery_pct >= 60`, giallo se `>= 30`, rosso se `< 30`. Il valore è simulato nel seed (50-100%) e nella produzione sarebbe aggiornato dai sensori fisici del mezzo. La funzione di suggerimento mezzo in **search.tsx** usa lo stesso campo per filtrare i veicoli inadeguati alla distanza selezionata.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
vehicles	SELECT	PostgreSQL → schema: public → tabella: vehicles	id, battery_pct, type, status <i>battery_pct incluso in VehicleOut per ogni richiesta</i>

Come Verificare

SELECT battery_pct FROM vehicles WHERE id=X: confrontare con il valore mostrato nell'app. UPDATE vehicles SET battery_pct=10 WHERE id=X: l'indicatore nell'app deve diventare rosso.

Sblocco mezzo tramite QR code o app

Endpoint	Screen / Componente
POST /rides	(app)/scan.tsx + (app)/activate.tsx + (app)/vehicle-action.tsx

Descrizione Logica del Sistema

L'utente scansiona il QR code del mezzo (formato **SM-{id}**) con **scan.tsx**, oppure inserisce il codice manualmente in **activate.tsx**. La funzione **start_ride()** in **rides.py** verifica prima che l'utente non abbia già una corsa attiva (409 se presente). Controlla poi che il mezzo abbia **status='parked'** e **locked=False**: se bloccato o non disponibile restituisce 409. Imposta **vehicle.status='in_use'** e crea un record **Ride** con **status='active'** e il timestamp di partenza. Se esiste una prenotazione attiva per quel mezzo, viene annullata automaticamente.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
rides	INSERT	PostgreSQL → schema: public → tabella: rides	user_id, vehicle_id, vehicle_type, from_addr, to_addr, status, started_at <i>status='active', started_at=now() (server-side)</i>
vehicles	UPDATE	PostgreSQL → schema: public → tabella: vehicles	status <i>status='in_use' al momento dello sblocco</i>
reservations	UPDATE	PostgreSQL → schema: public → tabella: reservations	status <i>status='used' se prenotazione attiva presente</i>

Come Verificare

Dopo start_ride: SELECT status FROM vehicles WHERE id=X → 'in_use'. SELECT status FROM rides WHERE id=Y → 'active'. Tentare sblocco con locked=true → 409.

Gestione metodo di pagamento

Endpoint	Screen / Componente
GET /payment-methods POST /payment-methods DELETE /payment-methods/{id}	(app)/wallet.tsx

Descrizione Logica del Sistema

La funzione **list_methods()** in *payment.py* restituisce i metodi di pagamento dell'utente ordinati con il default per primo. Quando si aggiunge un nuovo metodo con **add_method()**, se viene marcato come default il sistema aggiorna preventivamente tutti gli altri metodi dell'utente impostando **is_default=False**, garantendo che ci sia sempre un solo default. La cancellazione con **delete_method()** verifica che il metodo appartenga all'utente corrente prima di eliminarlo, impedendo cancellazioni non autorizzate. Il metodo default viene selezionato automaticamente in *ride-payment.tsx*.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
payment_methods	SELECT	PostgreSQL → schema: public → tabella: payment_methods	id, user_id, kind, label, last4, is_default Filtro: user_id = utente corrente, ORDER BY is_default DESC
payment_methods	INSERT	PostgreSQL → schema: public → tabella: payment_methods	user_id, kind, label, last4, is_default Aggiunta nuovo metodo di pagamento
payment_methods	UPDATE	PostgreSQL → schema: public → tabella: payment_methods	is_default Reset is_default=false su tutti i metodi prima di impostare il nuovo default
payment_methods	DELETE	PostgreSQL → schema: public → tabella: payment_methods	id Verifica user_id prima della cancellazione

Come Verificare

Aggiungere due metodi marcando il secondo come default: SELECT is_default FROM payment_methods WHERE user_id=X → solo uno deve essere true.

Pausa corsa

Endpoint	Screen / Componente
PATCH /rides/{id}/pause	(app)/active-ride.tsx

Descrizione Logica del Sistema

La funzione **toggle_pause()** in *rides.py* alterna lo stato della corsa tra 'active' e 'paused'. Alla messa in pausa, salva il timestamp nel campo **orario_inizio_pausa**. Alla ripresa, calcola i secondi trascorsi sottraendo quel timestamp dall'ora corrente e li accumula in **pausa_secondi_accumulati**, poi azzerava **orario_inizio_pausa**. Il mezzo rimane fisicamente in **status='in_use'** durante tutta la pausa: nessun altro utente può prenderlo. Nel prototipo il costo finale non deduce i secondi di pausa (sono però tracciati per audit); in produzione il calcolo del costo sottrarrebbe **pausa_secondi_accumulati** dalla durata totale.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
rides	UPDATE	PostgreSQL → schema: public → tabella: rides	status, orario_inizio_pausa, pausa_secondi_accumulati Pausa: status='paused', orario_inizio_pausa=now. Ripresa: status='active', pausa_secondi_accumulati+=elapsed, orario_inizio_pausa=null

Come Verificare

Mettere in pausa: SELECT status, orario_inizio_pausa FROM rides WHERE id=X → 'paused' e timestamp. Riprendere: pausa_secondi_accumulati deve essere >0.

Creazione profilo utente

Endpoint	Screen / Componente
POST /auth/register	(auth)/register.tsx

Descrizione Logica del Sistema

La funzione **register()** in *auth.py* verifica prima che l'email non sia già presente nel DB (409 se esiste). Determina il ruolo finale dal campo **data.role**: accetta solo valori in ROLES (UTENTE, OPERATORE, AMMINISTRAZIONE), altrimenti assegna UTENTE di default. Crea il record **User** con la password cifrata da **hash_password()** (bcrypt). Dopo il flush per ottenere l'id, crea il profilo esteso: **OperatoreProfile** o **AmministrazioneProfile** per i ruoli speciali, oppure un messaggio di benvenuto per gli utenti normali. La risposta include già il JWT per il login automatico post-registrazione.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
users	INSERT	PostgreSQL → schema: public → tabella: users	name, surname, email, phone, password_hash, provider, role, account_status, created_at <i>password_hash</i> cifrato con <i>bcrypt</i> ; <i>account_status</i> ='ATTIVO'
operatore_profili	INSERT	PostgreSQL → schema: public → tabella: operatore_profili	user_id, zona_competenza <i>Solo se role='OPERATORE'</i>
amministrazione_profili	INSERT	PostgreSQL → schema: public → tabella: amministrazione_profili	user_id, ente_appartenenza <i>Solo se role='AMMINISTRAZIONE'</i>
messages	INSERT	PostgreSQL → schema: public → tabella: messages	user_id, type, title, body <i>Messaggio benvenuto solo per role='UTENTE'</i>

Come Verificare

SELECT password_hash FROM users WHERE email='test@x.com' → deve iniziare con '\$2b\$' (bcrypt). Registrare stessa email due volte → 409.

Visualizzazione aree non accessibili

Endpoint	Screen / Componente
GET /aree-restrizione GET /aree-restrizione/verifica?lat=&lng;=&vehicle;_type=	(app)/index.tsx (overlay mappa)

Descrizione Logica del Sistema

La funzione **list_aree()** in *restrizioni.py* restituisce tutte le aree di restrizione a qualsiasi utente autenticato, poiche sono informazioni pubbliche. L'app renderizza cerchi colorati sulla mappa in base al tipo: rosso per NO_GO, arancio per ZTL, giallo per NO_PARKING. La funzione **verifica_posizione()** calcola per ogni area attiva la distanza euclidea tra la posizione inviata e il centro dell'area con la formula di Haversine (**_meters()**): se la distanza $\leq$ radius_m l'area e violata. Lo stesso controllo e integrato nel routing tramite **_aree_attraversate()**.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
aree_restrizio ne	SELECT	PostgreSQL → schema: public → tabella: aree_restrizione	id, nome, tipo, lat, lng, radius_m, vehicle_types, orario, attiva <i>Lettura completa per overlay mappa e verifica posizione</i>

Come Verificare

Creare area NO_GO (tipo='NO_GO', attiva=true): deve apparire sulla mappa. GET /aree-restrizione/verifica con coordinate dentro il raggio → restricted=true.

Segnalazione problematiche percorso

Endpoint	Screen / Componente
POST /reports GET /reports	(app)/report.tsx + (app)/reports-history.tsx

Descrizione Logica del Sistema

La funzione **create_report()** in *reports.py* crea una **Segnalazione** con stato iniziale 'APERTA'. Accetta fino a 3 allegati fotografici come array JSON di stringhe base64 nel campo **attachments**. Le coordinate GPS (**gps_lat**, **gps_lng**) vengono rilevate automaticamente dal dispositivo e incluse per geolocalizzare il problema. La stessa funzione inserisce un messaggio di conferma nella inbox dell'utente. **list_reports()** restituisce solo le segnalazioni dell'utente corrente (filtrate per **user_id**), visibili in **reports-history.tsx**. L'operatore vede le segnalazioni di tipo MALFUNZIONAMENTO tramite endpoint separato.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
segnalazioni	INSERT	PostgreSQL → schema: public → tabella: segnalazioni	user_id, ride_id, category, description, tipo, gravita, stato, gps_lat, gps_lng, attachments, created_at <i>stato='APERTA'; attachments = array JSON di immagini base64</i>
segnalazioni	SELECT	PostgreSQL → schema: public → tabella: segnalazioni	id, category, description, stato, created_at <i>Filtro: user_id = utente corrente</i>
messages	INSERT	PostgreSQL → schema: public → tabella: messages	user_id, type, title, body <i>type='system', conferma ricezione segnalazione</i>

Come Verificare

SELECT stato FROM segnalazioni WHERE user_id=X ORDER BY created_at DESC LIMIT 1 → 'APERTA'. SELECT attachments → deve essere un array JSON valido.

Pagamento corsa

Endpoint	Screen / Componente
POST /rides/{id}/end GET /wallet	(app)/ride-payment.tsx + (app)/payment-receipt.tsx

Descrizione Logica del Sistema

Il pagamento avviene all'interno di **end_ride()** in *rides.py*: non esiste un endpoint di pagamento separato. Il costo è calcolato server-side (**unlock_fee + minuti * price_per_min**) per impedire alterazioni lato client. Il saldo in **users.balance** viene decrementato e la transazione registrata con importo negativo in **wallet_transactions**. La schermata **ride-payment.tsx** mostra prima una stima con opzione per inserire un codice promo; dopo la conferma la schermata **payment-receipt.tsx** mostra il riepilogo finale con km, durata, costo e punti guadagnati. Il wallet aggiornato è leggibile tramite **get_wallet()** in *wallet.py*.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
rides	UPDATE	PostgreSQL → schema: public → tabella: rides	cost, km, minutes, points, status, ended_at Costo definitivo calcolato server-side da <i>end_ride()</i>
users	UPDATE	PostgreSQL → schema: public → tabella: users	balance, points <i>balance -= cost; points += POINTS_PER_RIDE (10 base + 5 eco)</i>
wallet_transactions	INSERT	PostgreSQL → schema: public → tabella: wallet_transactions	user_id, type, label, amount, created_at <i>type='charge', amount=-cost (valore negativo)</i>

Come Verificare

SELECT cost FROM rides WHERE id=X → importo calcolato server. SELECT balance FROM users WHERE id=Y → decrementato. SELECT amount FROM wallet_transactions ORDER BY created_at DESC LIMIT 1 → negativo.

Monitoraggio frequenza utilizzo mezzi

Endpoint	Screen / Componente
GET /amministrazione/monitoraggio/frequenza?vehicle_type=&from;_date=&to;_date=	(admin)/monitoraggio.tsx

Descrizione Logica del Sistema

La funzione **monitoraggio_frequenza()** in *amministrazione.py* valida il tipo di mezzo e converte le date ISO in oggetti datetime. La funzione helper **_count_corse()** esegue un COUNT sulla tabella **rides** filtrando per vehicle_type, status='completed' e l'intervallo temporale. I dati sono aggregati e anonimi: nessun riferimento a user_id nel risultato (GDPR). La funzione **utilizzo_per_tipo()** usa GROUP BY su vehicle_type per mostrare la distribuzione completa di corse, km e minuti per ogni tipo di mezzo.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
rides	SELECT	PostgreSQL → schema: public → tabella: rides	vehicle_type, status, km, minutes, started_at Aggregazione: COUNT(*), SUM(km), SUM(minutes) GROUP BY vehicle_type

Come Verificare

Inserire 5 corse completate in scooter nel DB. GET
/amministrazione/monitoraggio/frequenza?vehicle_type=scooter → totale_corse=5.

Segnalazione manutenzioni urbane

Endpoint	Screen / Componente
POST /amministrazione/segnala-zona	(admin)/segnala-zona.tsx

Descrizione Logica del Sistema

La funzione **segnala_zona()** in *amministrazione.py* permette all'Amministrazione di segnalare aree con manutenzioni urbane. Se le coordinate GPS non sono fornite direttamente, chiama **_geocode_first()** (Nominatim OSM) per risolvere il nome della zona in latitudine e longitudine. Crea una **Segnalazione** con **tipo='PERCORSO'** e **category='MANUTENZIONE_URBANA'**, distinguendola dalle segnalazioni utente (tipo='MALFUNZIONAMENTO'). L'operatore le vedrà nella propria dashboard filtrate per tipo.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
segnalazioni	INSERT	PostgreSQL → schema: public → tabella: segnalazioni	user_id, category, description, tipo, gravita, stato, gps_lat, gps_lng, zona, valida_dal, valida_al, created_at <i>tipo='PERCORSO', category='MANUTENZIONE_URBANA', stato='APERTA'</i>

Come Verificare

SELECT tipo, category FROM segnalazioni ORDER BY created_at DESC LIMIT 1 → tipo='PERCORSO', category='MANUTENZIONE_URBANA'. gps_lat/gps_lng non null (geocodificati).

Tratte più utilizzate

Endpoint	Screen / Componente
GET /amministrazione/tratte/frequenza?vehicle_type=&from;_date=&to;_date=	(admin)/tratte-piu-utilizzate.tsx

Descrizione Logica del Sistema

La funzione **tratte_frequenza()** in *amministrazione.py* usa la helper **_top_tratte()** che esegue una query con GROUP BY sulla coppia (**from_addr**, **to_addr**) e ordina per COUNT discendente. La funzione **tratte_piu_usate()** (endpoint senza filtri) usa invece un **Counter** Python applicato a tutti i record per calcolare la distribuzione globale. Entrambe operano solo su corse con status='completed'. I dati servono all'Amministrazione per pianificare manutenzione stradale nelle zone più frequentate.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
rides	SELECT	PostgreSQL → schema: public → tabella: rides	from_addr, to_addr, vehicle_type, status, started_at GROUP BY (from_addr, to_addr) ORDER BY COUNT DESC

Come Verificare

Inserire 5 corse con from='A' to='B' e 2 con from='C' to='D'. GET /amministrazione/tratte/frequenza: primo risultato corse=5 (tratta A→B).

Configurazione aree con restrizioni

Endpoint	Screen / Componente
POST /aree-restrizione/configura PATCH /aree-restrizione/{id} DELETE /aree-restrizione/{id}	(admin)/inserisci-area-restrizione.tsx

Descrizione Logica del Sistema

La funzione **configura_area()** in *restrizioni.py* crea una nuova zona di restrizione. Se le coordinate lat/Ing sono già note (utente ha cliccato sulla mappa con autocomplete integrato), le usa direttamente; altrimenti chiama **_geocode_first()** per ottenerle dall'indirizzo testuale. L'area viene inserita con **attiva=True** e **created_by=user.id** per tracciabilità. Solo il ruolo AMMINISTRAZIONE può creare, modificare (funzione **aggiorna_area()**) ed eliminare (**elimina_area()**) le aree. Immediatamente dopo la creazione, l'area compare sulla mappa utente e nel sistema di routing che la esclude dai percorsi consigliati.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
aree_restrizio ne	INSERT	PostgreSQL → schema: public → tabella: aree_restrizione	nome, tipo, lat, lng, radius_m, vehicle_types, note, valida_dal, valida_al, attiva, created_by, created_at <i>attiva=true, created_by=admin.id per audit</i>
aree_restrizio ne	UPDATE	PostgreSQL → schema: public → tabella: aree_restrizione	nome, tipo, lat, lng, radius_m, attiva, note <i>Solo ruolo AMMINISTRAZIONE via PATCH</i>
aree_restrizio ne	DELETE	PostgreSQL → schema: public → tabella: aree_restrizione	id <i>Solo ruolo AMMINISTRAZIONE via DELETE</i>

Come Verificare

INSERT area NO_GO: GET /aree-restrizione → deve includerla. GET /route/options su percorso che la attraversa → restricted=true. Tentare DELETE con token utente normale → 403.

Report aggregati sulla mobilita

Endpoint	Screen / Componente
GET /amministrazione/report/mobilita GET /amministrazione/report	(admin)/report-mobilita.tsx

Descrizione Logica del Sistema

La funzione **report_mobilita()** in *amministrazione.py* combina **_count_corse()** e **_top_tratte()** in un unico endpoint. La funzione **report_aggregato()** e piu completa: calcola totale corse, km, minuti, km medi per corsa, distribuzione per tipo di mezzo, mezzi attivi (esclude quelli in 'maintenance') e segnalazioni aperte. Tutti i dati sono aggregati e anonimi: nessun dato personale e esposto in accordo con GDPR. Le query usano **func.count** e **func.sum** di SQLAlchemy per massimizzare l'efficienza.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
rides	SELECT	PostgreSQL → schema: public → tabella: rides	vehicle_type, status, km, minutes, started_at Aggregazione: tot_corse, tot_km, tot_min, km_medi
vehicles	SELECT	PostgreSQL → schema: public → tabella: vehicles	status COUNT mezzi_attivi: WHERE status != 'maintenance'
segnalazioni	SELECT	PostgreSQL → schema: public → tabella: segnalazioni	stato COUNT segnalazioni_aperte: WHERE stato='APERTA'

Come Verificare

GET /amministrazione/report: mezzi_attivi deve escludere quelli con status='maintenance'.
segnalazioni_aperte = SELECT COUNT(*) FROM segnalazioni WHERE stato='APERTA'.

Registrazione Amministrazione Pubblica

Endpoint	Screen / Componente
POST /auth/register {role:'AMMINISTRAZIONE'}	(auth)/register.tsx

Descrizione Logica del Sistema

Stessa funzione **register()** in *auth.py* usata per tutti i ruoli. Quando `role='AMMINISTRAZIONE'`, dopo aver creato il record **User**, la funzione crea un profilo esteso in **amministrazione_profilo** con il campo **ente_appartenenza** (es. 'Comune di Bari'). A differenza degli utenti normali, non viene inviato il messaggio di benvenuto. Il JWT restituito contiene `role='AMMINISTRAZIONE'`, che sblocca l'accesso a tutte le route `/amministrazione/*` protette da **require_role('AMMINISTRAZIONE')** in *deps.py*.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
users	INSERT	PostgreSQL → schema: public → tabella: users	name, email, password_hash, role, account_status <i>role='AMMINISTRAZIONE'; account_status='ATTIVO'</i>
amministrazione_profilo	INSERT	PostgreSQL → schema: public → tabella: amministrazione_profilo	user_id, ente_appartenenza <i>Profilo 1:1 creato dopo db.flush()</i>

Come Verificare

SELECT role FROM users WHERE email='admin@test.it' → 'AMMINISTRAZIONE'.
SELECT ente_appartenenza FROM amministrazione_profilo WHERE user_id=X → valore inserito.

Login Amministrazione Pubblica

Endpoint	Screen / Componente
POST /auth/login	(auth)/login.tsx

Descrizione Logica del Sistema

Il login dell'Amministrazione usa lo stesso endpoint **login()** in *auth.py* condiviso con tutti i ruoli. Il JWT generato da **create_access_token()** include il claim **role='AMMINISTRAZIONE'**. L'app legge questo claim e reindirizza al tab **(admin)/**. La funzione **require_role()** in *deps.py* protegge ogni endpoint */amministrazione/**: recupera l'utente dal DB tramite il JWT, controlla che **user.role == 'AMMINISTRAZIONE'** e lancia 403 in caso contrario.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
users	SELECT	PostgreSQL → schema: public → tabella: users	id, email, password_hash, role, account_status Verifica credenziali e ruolo. Nessuna scrittura.

Come Verificare

Decodificare JWT con JWT_SECRET → claim 'role'='AMMINISTRAZIONE'. Chiamare GET */amministrazione/report* con token utente normale → 403 Forbidden.

Registrazione Operatore

Endpoint	Screen / Componente
POST /auth/register {role:'OPERATORE'}	(auth)/register.tsx

Descrizione Logica del Sistema

Identica struttura di **register()** in *auth.py*, ma quando `role='OPERATORE'` viene creato un profilo **OperatoreProfile** con il campo **zona_competenza** (area geografica di responsabilit ). Questo profilo   separato dal record User (class table inheritance): permette di aggiungere attributi specifici dell'operatore senza modificare la tabella users principale. Il JWT con `role='OPERATORE'` sblocca tutte le route `/operatore/*` e la web dashboard **VistaOperatore.tsx**.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
users	INSERT	PostgreSQL → schema: public → tabella: users	name, email, password_hash, role, account_status <i>role='OPERATORE', account_status='ATTIVO'</i>
operatore_profili	INSERT	PostgreSQL → schema: public → tabella: operatore_profili	user_id, zona_competenza <i>Profilo 1:1 creato dopo db.flush(), before db.commit()</i>

Come Verificare

SELECT role FROM users WHERE email='op@test.it' → 'OPERATORE'. SELECT zona_competenza FROM operatore_profili WHERE user_id=X → valore inserito.

Notifiche aree a diversa disponibilit 

Endpoint	Screen / Componente
GET /operatore/aree-densita WS /ws/notifiche?token=	(operatore)/disponibilit�-aree.tsx

Descrizione Logica del Sistema

La funzione **aree_densita()** in *operatore.py* conta i veicoli con status='parked' AND locked=False all'interno del raggio di ogni area di parcheggio, usando un'approssimazione in gradi decimali ($\text{deg} = \text{radius_m} / 111000$). Il risultato classifica ogni area in quattro stati: **VUOTA** (0 mezzi), **CRITICA** (1-2), **OK** (3-5), **PIENA** (≥ 6). Parallelamente, il WebSocket **notifiche_operatore()** in *realtime.py* invia ogni 5 secondi gli avvisi per le aree con densit  BASSA (< 3 mezzi), consentendo monitoraggio continuo senza refresh manuale.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
vehicles	SELECT	PostgreSQL → schema: public → tabella: vehicles	status, locked, lat, lng Filtro: status='parked' AND locked=false per conteggio per area
parking_areas	SELECT	PostgreSQL → schema: public → tabella: parking_areas	id, name, lat, lng, radius_m, capacity Tutte le aree per calcolo densit�

Come Verificare

Con 0 veicoli in un'area: GET /operatore/aree-densita → stato='VUOTA'. Con 7 veicoli: stato='PIENA'. WebSocket: ricevere JSON ogni 5s con avvisi per aree critiche.

Conoscenza malfunzionamenti mezzi

Endpoint	Screen / Componente
GET /operatore/segnalazioni	(operatore)/malfunzionamenti-mezzi. tsx + (operatore)/segnalazione-dett aglio.tsx

Descrizione Logica del Sistema

La funzione **segnalazioni_mezzi()** in *operatore.py* interroga la tabella **segnalazioni** filtrando solo quelle con **tipo='MALFUNZIONAMENTO'** e **stato='APERTA'**. Il filtro è applicato lato backend (non modificabile dal client). I risultati vengono ordinati per gravità decrescente (ALTA prima) usando il dizionario **_GRAVITA_ORD**, poi per id decrescente. Il dettaglio di ogni segnalazione in **segnalazione-dettaglio.tsx** mostra la posizione GPS su mappa e consente all'operatore di procedere alla chiusura.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
segnalazioni	SELECT	PostgreSQL → schema: public → tabella: segnalazioni	id, tipo, gravita, stato, gps_lat, gps_lng, description, created_at <i>Filtro: tipo='MALFUNZIONAMENTO' AND stato='APERTA'; ORDER BY gravita ASC, id DESC</i>

Come Verificare

INSERT segnalazione tipo='PERCORSO': NON deve apparire in GET /operatore/segnalazioni. INSERT tipo='MALFUNZIONAMENTO' gravita='ALTA': deve apparire per prima nella lista.

Posizione mezzo a fine corsa

Endpoint	Screen / Componente
GET /operatore/mezzi-rilascio	(operatore)/mezzi-fine-corsa.tsx

Descrizione Logica del Sistema

La funzione **mezzi_rilascio()** in *operatore.py* restituisce tutti i veicoli con **status='parked'**, inclusi quelli con **locked=True**. Questo è il comportamento opposto rispetto alla vista utente (*list_vehicles* con *only_available=True* che esclude i veicoli bloccati): l'operatore deve vedere DOVE si trovano fisicamente tutti i mezzi parcheggiati, anche quelli bloccati da lui stesso, per pianificare interventi sul campo. Le coordinate reali (lat, lng) mostrano la posizione esatta nell'area di sosta.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
vehicles	SELECT	PostgreSQL → schema: public → tabella: vehicles	id, name, type, lat, lng, status, locked, battery_pct Filtro: <i>status='parked'</i> (include <i>locked=true</i>); <i>ORDER BY id</i>

Come Verificare

SELECT COUNT(*) FROM vehicles WHERE status='parked' AND locked=true → N. GET /operatore/mezzi-rilascio → deve includere N veicoli bloccati. GET /vehicles?only_available=true → N veicoli bloccati ASSENTI.

Tracciamento posizione in tempo reale

Endpoint	Screen / Componente
WS /ws/mezzi/{vehicle_id}?token=	(operatore)/tracciamento-mezzi.tsx

Descrizione Logica del Sistema

Il WebSocket **track_vehicle()** in *realtime.py* autentica l'operatore controllando il JWT passato come query param (gli header non sono pratici nei WebSocket). La funzione **_auth()** decodifica il token, recupera l'utente dal DB e verifica che `role='OPERATORE'` e `account_status='ATTIVO'`. Ogni 2 secondi il loop aggiorna le coordinate del mezzo nel DB simulando un drift GPS di ± 0.0002 gradi (~22 m) e invia il payload JSON `{vehicle_id, lat, lng, battery_pct, status, locked}` al client. La disconnessione del client viene gestita con **WebSocketDisconnect** senza errori.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
vehicles	SELECT	PostgreSQL → schema: public → tabella: vehicles	id, lat, lng, battery_pct, status, locked Lettura ogni 2s per il payload WebSocket
vehicles	UPDATE	PostgreSQL → schema: public → tabella: vehicles	lat, lng Aggiornamento coordinate ogni 2s (simulazione movimento)
users	SELECT	PostgreSQL → schema: public → tabella: users	id, role, account_status Solo all'apertura della connessione per autenticazione

Come Verificare	SELECT lat, lng FROM vehicles WHERE id=X; attendere 4s; SELECT lat, lng again → valori diversi. WebSocket senza token → close code 4401.
-----------------	--

Chiusura segnalazione

Endpoint	Screen / Componente
PATCH /segnalazioni/{id}/chiudi	(operatore)/chiudi-segnalazioni.tsx + (operatore)/segnalazione-dettagli.tsx

Descrizione Logica del Sistema

La funzione **chiudi_segnalazione()** in *segnalazioni.py* richiede ruolo OPERATORE. Verifica che la segnalazione esista e che lo stato sia 'APERTA' (409 se già chiusa, per idempotenza). Imposta **stato='CHIUSA'** e registra **closed_at=now()** per il tracciamento temporale delle risoluzioni. Inserisce un messaggio nella inbox dell'utente che aveva aperto la segnalazione, notificandolo della risoluzione. La lettura delle segnalazioni è permessa anche all'Amministrazione (require_role con più ruoli), ma la chiusura è esclusiva dell'Operatore.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
segnalazioni	UPDATE	PostgreSQL → schema: public → tabella: segnalazioni	stato, closed_at stato='CHIUSA', closed_at=now() UTC
messages	INSERT	PostgreSQL → schema: public → tabella: messages	user_id, type, title, body Notifica all'utente: 'Segnalazione risolta'

Come Verificare

PATCH /segnalazioni/{id}/chiudi: SELECT stato, closed_at FROM segnalazioni WHERE id=X → 'CHIUSA' e timestamp. Richiamare PATCH → 409 Conflict.

Assegnazione automatica bonus

Endpoint	Screen / Componente
GET /operatore/bonus/config POST /operatore/bonus	(operatore)/assegna-bonus.tsx

Descrizione Logica del Sistema

La funzione **assegna_bonus()** in *operatore.py* itera su tutti gli utenti attivi con `role='UTENTE'`. Per ognuno conta le corse completate con una query COUNT. Se il conteggio raggiunge la soglia configurata dall'operatore (**data.soglia_corse**), incrementa **user.points** del bonus specificato e inserisce un messaggio di notifica di tipo 'promo' nella inbox dell'utente. La configurazione di default (5 corse / 50 punti) è restituita da **get_bonus_config()** e può essere personalizzata ad ogni invocazione. La funzione restituisce la lista degli `user_id` premiati per audit.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
users	SELECT	PostgreSQL → schema: public → tabella: users	id, role, account_status, points <i>Tutti gli utenti ATTIVI con role='UTENTE'</i>
rides	SELECT	PostgreSQL → schema: public → tabella: rides	user_id, status <i>COUNT per user WHERE status='completed'</i>
users	UPDATE	PostgreSQL → schema: public → tabella: users	points <i>points += punti_bonus per ogni utente idoneo</i>
messages	INSERT	PostgreSQL → schema: public → tabella: messages	user_id, type, title, body <i>type='promo', notifica bonus a ogni utente premiato</i>

Come Verificare

Utente con 5 corse completate e soglia=5: SELECT points FROM users WHERE id=X prima → N; dopo POST /operatore/bonus → N+50. Utente con 2 corse: points invariati.

Sospensione o blocco account utente

Endpoint	Screen / Componente
GET /operatore/utenti POST /operatore/utenti/{id}/stato	(operatore)/blocco-utenti.tsx

Descrizione Logica del Sistema

La funzione **cambia_stato_utente()** in *operatore.py* modifica **account_status** dell'utente target (ATTIVO/SOSPESO/BLOCCATO). Salva la transizione di stato nel log di audit **AzioneOperatoreLog** con **operatore_id**, **utente_id**, **motivo** e **dettaglio** (es. 'ATTIVO → BLOCCATO'). Inserisce una notifica personalizzata nella inbox dell'utente con il motivo. Al prossimo login, **login()** in *auth.py* verifica **account_status** e restituisce 403 per account SOSPESO/BLOCCATO. Se l'utente ha una sessione attiva, ogni richiesta autenticata riceve 403 e l'app esegue il logout immediato.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
users	UPDATE	PostgreSQL → schema: public → tabella: users	account_status Nuovo stato: ATTIVO SOSPESO BLOCCATO
azioni_operatore_log	INSERT	PostgreSQL → schema: public → tabella: azioni_operatore_log	operatore_id, utente_id, azione, motivo, dettaglio, timestamp Audit trail immutabile di ogni cambio stato
messages	INSERT	PostgreSQL → schema: public → tabella: messages	user_id, type, title, body type='alert', notifica motivata all'utente

Come Verificare

SELECT account_status FROM users WHERE id=X → 'BLOCCATO'. SELECT azione FROM azioni_operatore_log WHERE utente_id=X → 'CAMBIO_STATO_ACCOUNT'. Login con quell'utente → 403.

Blocco remoto mezzo

Endpoint	Screen / Componente
POST /operatore/mezzi/{id}/blocco {locked:true false}	(operatore)/blocco-remoto.tsx + VistaOperatore.tsx (dashboard)

Descrizione Logica del Sistema

La funzione **blocco_remoto()** in *operatore.py* gestisce blocco e sblocco. Al blocco, cerca prima se c'è una corsa attiva sul mezzo: se esiste, la imposta su 'interrupted' e invia un avviso all'utente in corsa. Poi imposta **vehicle.locked=True** e **vehicle.status='parked'**. Se il mezzo era in movimento (fuori da qualsiasi area di sosta), la funzione **_nearest_area()** trova l'area più vicina e **rand_coord_in_area()** riposiziona il mezzo al suo interno. Ogni azione è registrata in **AzioneOperatoreLog**. Il mezzo bloccato non compare nella mappa utente (filtro locked=False).

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
vehicles	UPDATE	PostgreSQL → schema: public → tabella: vehicles	locked, status, lat, lng <i>locked=true/false; status='parked'; lat/lng aggiornati se fuori area</i>
rides	UPDATE	PostgreSQL → schema: public → tabella: rides	status, ended_at <i>status='interrupted', ended_at=now() se corsa attiva al blocco</i>
azioni_operatore_log	INSERT	PostgreSQL → schema: public → tabella: azioni_operatore_log	operatore_id, azione, dettaglio, timestamp <i>BLOCCO_REMOTO_MEZZO o SBLOCCO_REMOTO_MEZZO + ride_id se corsa interrotta</i>
messages	INSERT	PostgreSQL → schema: public → tabella: messages	user_id, type, title, body <i>Avviso all'utente in corsa: 'Corsa interrotta'</i>

Come Verificare

SELECT locked, status FROM vehicles WHERE id=X → true, 'parked'. Con corsa attiva:
SELECT status FROM rides WHERE vehicle_id=X AND status='interrupted' → 1 row. GET
/vehicles?only_available=true → mezzo assente.

Login Operatore

Endpoint	Screen / Componente
POST /auth/login	(auth)/login.tsx + FormAutenticazione.tsx (dashboard web)

Descrizione Logica del Sistema

Identico a tutti gli altri login: la funzione **login()** in *auth.py* genera un JWT con `role='OPERATORE'`. L'app mobile legge il ruolo e reindirizza al tab **(operatore)/**. La web dashboard (**VistaOperatore.tsx** via **FormAutenticazione.tsx**) salva il token in `localStorage` e lo include nell'header di tutte le chiamate alle API `/operatore/*`. La funzione **require_role('OPERATORE')** in *deps.py* protegge ogni endpoint: recupera l'utente dal DB tramite il JWT e verifica che `user.role == 'OPERATORE'`, restituendo 403 in caso contrario.

Percorso Dati nel Database

Tabella	Operazione	Percorso DB completo	Colonne · Note
users	SELECT	PostgreSQL → schema: public → tabella: users	id, email, password_hash, role, account_status Verifica credenziali e <code>account_status</code> . Nessuna scrittura.

Come Verificare

JWT decodificato → claim `'role'='OPERATORE'`. GET `/operatore/flotta` con token utente normale → 403 Forbidden. GET `/operatore/flotta` con token operatore → 200 OK.

Schema Database — Riepilogo Completo

Tabella	Colonne	User Stories
users	id · name · surname · email · phone · password_hash · provider · points · balance · role · account_status · created_at	UT.10/15 · OP.08/09 · AP.06/07
operatore_profili	id · user_id(FK) · zona_competenza · matricola	OP.01
amministrazione_profilo	id · user_id(FK) · ente_appartenenza · codice_ente	AP.06
vehicles	id · name · model · type · lat · lng · battery_pct · status · unlock_fee · price_per_min · locked	UT.01/07/11/12 · OP.04/05/09
parking_areas	id · name · address · lat · lng · radius_m · capacity · occupied	UT.01/05 · OP.02
rides	id · user_id · vehicle_id · vehicle_type · from_addr · to_addr · km · minutes · cost · points · status · started_at · ended_at · orario_inizio_pausa · pausa_secondi_accumulati	UT.04/05/12/14/18 · AP.01/03/05
reservations	id · user_id · vehicle_id · created_at · expires_at · status	UT.02
wallet_transactions	id · user_id · type · label · amount · created_at	UT.13/18
payment_methods	id · user_id · kind · label · last4 · is_default	UT.13
promotions	id · code · title · body · reward · kind · expiry · used · total	UT.08
messages	id · user_id · type · title · body · read · created_at	UT.09 · OP.06/07/08/09
segnalazioni	id · user_id · ride_id · category · description · tipo · gravita · stato · gps_lat · gps_lng · attachments · zona · valida_dal · valida_al · created_at · closed_at	UT.17 · AP.02 · OP.03/06
aree_restrizione	id · nome · tipo · lat · lng · radius_m · vehicle_types · orario · attiva · note · valida_dal · valida_al · created_by · created_at	UT.16 · AP.04 · OP.02
azioni_operatore_log	id · operatore_id · utente_id · azione · motivo · dettaglio · timestamp	OP.08/09 (audit trail)